Constitutional Syscalls

Explicit Authority Contracts for System Interaction

As computation moves into virtualized, sandboxed, replayable, and distributed domains, that presumption becomes unstable.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Classical system calls fuse mechanism and jurisdiction: an operation and its permission are inseparable, presuming a privileged kernel as the sole authority over execution reality. As computation moves into virtualized, sandboxed, replayable, and distributed domains, that presumption becomes unstable.

This paper reconceptualizes the system call as a contracted authority crossing - a typed request mediated by explicit admissibility rules. A constitutional boundary separates what can be done from what may be done here and now. Each call is evaluated under a contract defining its semantics, authority requirements, and admissible outcomes: execution, simulation, deferral, or refusal. Refusal is not failure but a legitimate, typed result carrying structured explanation.

We formalize this model with a minimal admissibility function and extend it to multi-authority chains. Using graphical user interface (GUI) operations as a stress test, we demonstrate temporal portability, deterministic replay, and auditable refusal across heterogeneous authorities. Execution semantics evolve from trap to adjudication: the system no longer merely obeys instructions; it interprets them under contract.

---

1. From Trap to Adjudication

The classical syscall model treats a system call as a privileged trap into a kernel that owns execution reality. User code invokes; the kernel decides; the machine complies. Authority is implicit in position - kernel space is legitimate, user space is constrained.

This model has been remarkably effective. Yet it embeds a strong ontological assumption: that there exists a single jurisdiction grounding all legitimate system actions.

As execution environments diversify - virtual machines, WebAssembly runtimes, replay engines, distributed hosts - that presumption becomes unstable. The same syscall may be interpretable, simulatable, deferrable, or inadmissible depending on context.

The thesis of this paper is precise:

> System calls should be understood not as privileged traps, but as constitutional boundary crossings - requests adjudicated under explicit authority contracts.

This reframing moves execution semantics from trap to adjudication.

---

2. Mechanism and Jurisdiction

The instability arises from conflating two concerns:

Mechanism - how an operation could be performed

Jurisdiction - whether an operation is permitted under a specific authority and contract

Traditional OS design binds these together. If the kernel understands how to perform an operation, it is presumed authorized to do so.

In modern layered systems, this presumption fails. A replay engine may understand file I/O but refuse execution to preserve determinism. A sandbox may know how to open sockets but lack jurisdiction to permit it.

Separating mechanism from jurisdiction allows a precise formulation:

Admissibility is an explicit function, not an implicit assumption.

---

3. Redefining the System Call Boundary

We propose:

> A system call is a typed request to cross an authority boundary, governed by an explicit contract, whose outcome may be execution, simulation, deferral, or refusal.

Three consequences follow.

1. **Boundary-Defined Meaning** The syscall's semantics are defined at the boundary contract, not by a specific kernel implementation
2. **Contract-Stabilized Interpretation** Arguments and required authority levels are declared in versioned contracts

3. **Refusal as Correct Behavior** Refusal is a first-class, typed outcome with structured reason

Figure 1. Constitutional boundary and admissible outcomes

```
User Code -> [Authority Boundary]
                  v
        execute | simulate | defer | refuse
```

Each arrow represents a possible adjudication under contract. "Refuse" completes the set, closing open-ended failure into structured accountability. Outcomes are exclusive per boundary evaluation but may be sequenced across layered authorities.

---

4. Minimal Constitutional Formalism

We define:

```
Syscall := {
  op : OperationType,
  args : TypedArguments,
  required_authority : AuthorityLevel,
  contract_id : ContractIdentifier
}
```

```
Authority := {
  level : AuthorityLevel,
  capabilities : Set[OperationType],
  policies : Set[Constraints],
  contract_registry : Set[ContractIdentifier]
}
```

Define the admissibility function:

$$\text{Admissibility} : \text{Syscall} \times \text{Authority} \to \text{Outcome} \times \text{Reason}$$

where $\text{Outcome} \in \{\text{execute},\ \text{simulate},\ \text{defer},\ \text{refuse}\}$

Reason is structured metadata:

```
{"outcome":"refuse","reason":"insufficient_authority"}
```

Admissibility(S, A) yields:

- **execute** if authority level, capability, and policy constraints are satisfied
- **simulate** if execution is disallowed but simulation is contractually defined
- **defer** if temporal or environmental preconditions are unmet
- **refuse** otherwise

Refusal is not exceptional. It is constitutional.

Contract Provisioning

The `contract_registry` is provisioned externally - at runtime instantiation via trusted loader, cryptographic signature verification, or authority bootstrap configuration. Contracts are versioned and auditable; authorities may refuse calls with unsupported contract identifiers.

This prevents circular self-authorization.

---

5. Multi-Authority Composition

Layered systems introduce authority chains:

$$A_1 \to A_2 \to \cdots \to A_n$$

For example: Replay Engine -> Runtime -> Kernel -> Hardware.

Define sequential evaluation:

$$\text{Admissibility}^*(S,\ [A_1 \dots A_n])$$

```
for i from 1 to n:
    (outcome_i, reason_i) = Admissibility(S, A_i)
    if outcome_i != execute:
        return (outcome_i, reason_i)

return (execute, {"reason": "all_authorities_admit"})
```

Thus:

- Refusal by any authority terminates evaluation
- Simulation or deferral may occur at intermediate layers
- Execution requires unanimous admissibility

This model supports layered governance without ambiguity.

---

6. GUI as Stress Test

Graphical user interface operations require long-lived state, asynchronous events, and partial execution.

Under the classical trap model, GUI semantics accrete in user-space servers.

Under constitutional boundaries:

- A replay authority may simulate
- A sandbox may refuse
- A runtime may defer
- A kernel may execute

The same contract governs all layers.

The complexity is adjudicated, not obscured.

---

7. Relation to Prior Systems

Constitutional syscalls generalize:

- **Capability systems** - where authority is explicit and token-bound
- **Reference monitor theory** - where decisions are centralized and enforceable
- **seL4 and microkernels** - which separate mechanism from policy
- **WASI and WebAssembly runtimes** - which abstract host functionality via explicit host functions

The extension lies in elevating the decision function itself to a typed, inspectable object with structured outcomes.

---

8. Cost and Overhead

Explicit admissibility introduces cost:

- Additional boundary evaluation
- Structured logging
- Contract version negotiation
- Authority chain traversal

These costs resemble measured IPC and capability-check overheads in microkernel systems such as seL4.

The tradeoff is explicit adjudication for implicit assumption.

Where classical syscalls risk undefined or silent failure, constitutional syscalls incur overhead to ensure structured refusal and replayable decisions.

---

9. Architectural Consequences

The model enables:

- Kernel-optional primary authorities
- Deterministic replay with structured logs

- Multi-jurisdiction enforcement
- Explicit failure-closed semantics

It parallels invariant descent patterns: authority evaluation moves from advisory logic into substrate enforcement. A natural extension is to treat any reality-touching effect—GUI, I/O, time, networking—as a constitutional boundary crossing, keeping the computational core effect-pure.

---

10. Constitutional Logging and Replay

A minimal audit structure:

```
ConstitutionalLog := {
  syscall : Syscall,
  outcome : Outcome,
  reason : Reason,
  contract_id : ContractIdentifier,
  timestamp : Time
}
```

Such logs support deterministic replay, auditability, and governance traceability.

Future work includes partial static reasoning: verifying admissibility constraints from declared contracts prior to execution, analogous to type checking.

---

11. Conclusion

Classical systems treat syscalls as privileged traps.

Constitutional systems treat syscalls as governed authority crossings.

In constitutional execution environments, refusal is no longer an error - it is part of correct behavior.

By separating mechanism from jurisdiction and formalizing admissibility as a structured function, execution becomes accountable across layered authorities.

The machine does not merely execute. It adjudicates.

Corpus-Wide Reference Positioning

This paper argues that classical system calls fuse mechanism and jurisdiction: an operation and its permission are inseparable, presuming a privileged kernel as the sole authority over execution reality. Its nearest external comparison fields are security, formal, cps. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims.

The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Runtime monitors and fallback controllers only protect modeled safety envelopes and can fail through sensor error, timing assumptions, unsafe recovery, or omitted hazards. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Jerome H. Saltzer and Michael D. Schroeder. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63(9), 1975, pp. 1278-1308. DOI: 10.1109/PROC.1975.9939. https://doi.org/10.1109/PROC.1975.9939 **Relationship:** Foundational protection principles used as a comparison point for least authority, mediation, and inspectability.

[**E2**] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD dissertation, Johns Hopkins University, 2006. http://www.erights.org/talks/thesis/ **Relationship:** Predecessor object-capability treatment of authority and compositional security.

[**E3**] Gerwin Klein et al. “seL4: Formal Verification of an OS Kernel.” *SOSP ’09*, 2009, pp. 207-220. DOI: 10.1145/1629575.1629596. https://doi.org/10.1145/1629575.1629596 **Relationship:** Predecessor for machine-checked implementation correspondence and capability-based kernel design.

[**E4**] Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Peter G. Neumann. *An Introduction to CHERI*. University of Cambridge Technical Report UCAM-CL-TR-941, 2019. DOI: 10.48456/tr-941. cl.cam.ac.uk source **Relationship:** Adjacent capability architecture demonstrating unforgeable hardware authority and compartmentalization.

[**E5**] George C. Necula. “Proof-Carrying Code.” *POPL ’97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E6**] Xavier Leroy. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E7**] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” *POPL ’77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

Ableman corpus

[**A1**] Adam Ableman. *Effect Sovereignty*. Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality*. Ableman Corpus current edition; work identity `PAPER.PUB.18601631`;
source SHA-256
`6DE46F821594AB11BFB7F8373368C452BD89E5BD4EB172B88D9467E87CF74C33`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Integrity Enforcement Layer*. Ableman Corpus current edition; work identity

AMCS-LOCAL-WORK-505307E9E6571B63;
source SHA-256
F858AB32E0937B796353CAE573DD4183FDF4F179A3E1E7E40C4E0C6BF1AB0018.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Universal Governance Kernel (UGK).* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-1F9BFB2BF8370226;
source SHA-256
AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Ableman Corpus current edition; work identity PAPER.PUB.21095303;
source SHA-256
EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Governed Semantic Compilation.* Ableman Corpus current edition; work identity PAPER.PUB.18453681;
source SHA-256
5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Hybrid Semantic Compiler Architectures.* Ableman Corpus current edition; work identity PAPER.PUB.21228038;
source SHA-256
EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18737229